University of Toronto

CSC490

A2

Name	Student Number
Yusuf Moola	1007658777
Mikhail Skazhenyuk	1009376337
Rudraksh Monga	1008018342
Codi Lee	1007698608

1 Aspirational Datasets

If we had no restrictions, our ideal dataset for this project would cover every brand that appears in Formula 1 broadcasts with exhaustive annotations. Specifically, we would want:

- **Full race coverage** for multiple seasons, with every frame annotated for brand logos on cars with specific location, driver suits, track side banners, and virtual ads.
- **Schema:** `frame_id`, `timestamp`, `brand_id`, `bbox` (normalized), `occlusion_level`, `blur_score`, `context_type`, and `visibility_score`.
- **Brand identity:** Having access to the specific logos and variations brands will be putting on the car is important as checking for a handful of logos is easier than checking for all.
- Integration of other motor sports (IndyCar, MotoGP, Formula E) to improve generalization.
- Synthetic data, compositing official sponsor logos onto race frames at different scales, angles, and lighting, to cover edge cases.
- Documentation and knowledge on the source and whereabouts of the frames. Particularly metadata indicating what video (possibly from what part of the video) each frame originated so as to avoid data leakage in the train/test splits.

With such a dataset we could train models that not only detect logos but also predict quality of visibility (e.g., blurred vs. crisp, obstructed vs. clear). This would enable robust benchmarking across seasons and broadcast styles.

2 Reality Check

In practice, we do not have infinite data. Our primary dataset is the Roboflow F1 Logo Dataset, but we reviewed public alternatives and augmentation strategies. Table 1 summarizes realistic datasets we can use.

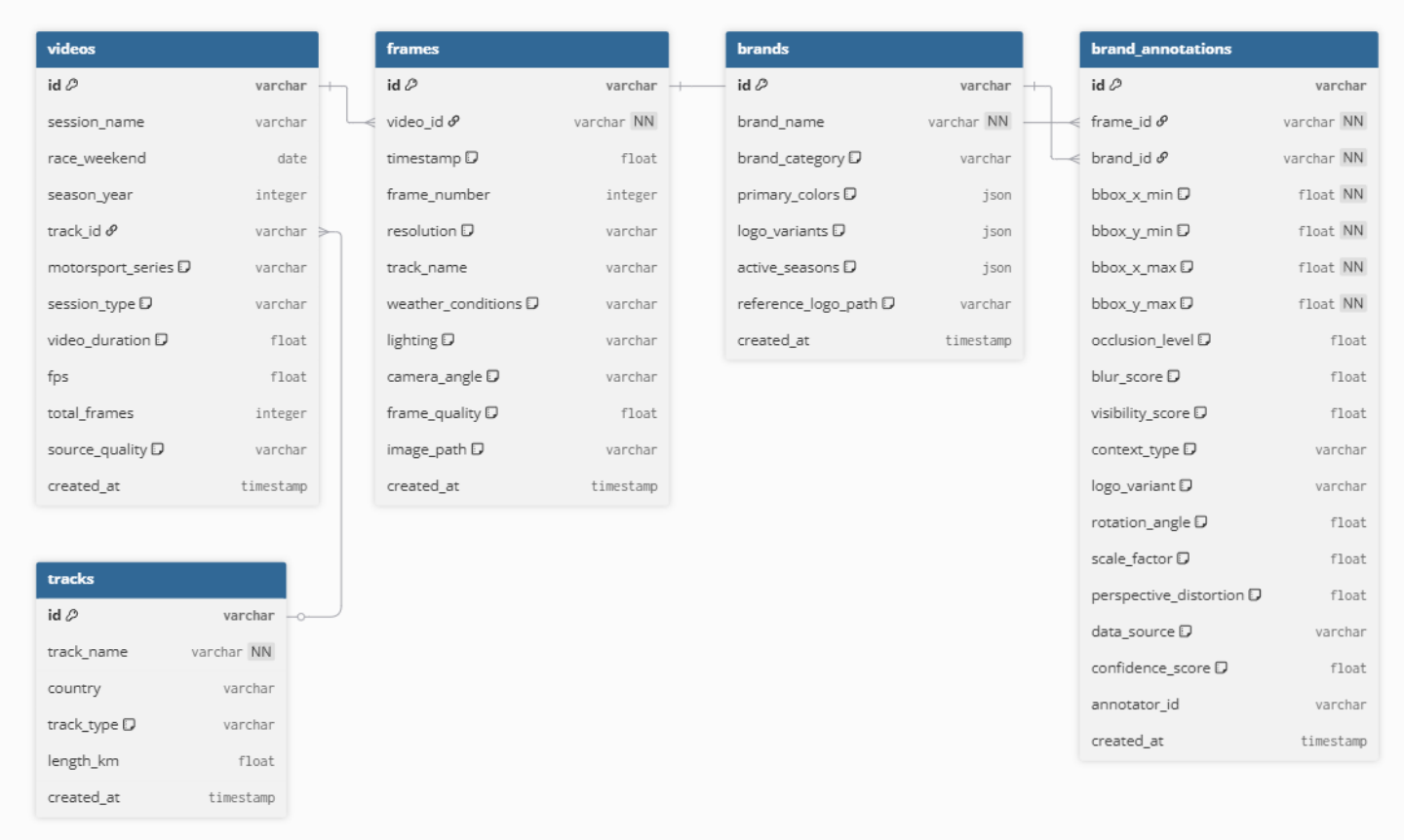

Figure 1: Ideal Data Schema

Table 1: *Candidate datasets for F1 sponsorship detection*

Relevant Item	**Description**	**Commentary**
Roboflow F1 Logo Dataset 1	∼3,049 labeled frames from F1 broadcasts, with bounding boxes for ∼86 brands.	Our primary dataset. Directly relevant, but relatively small compared to the diversity of F1 conditions.
QMUL OpenLogo 2	27k images , 352 logo classes from diverse domains.	Useful for pretraining to teach the model "what a logo looks like." Not F1-specific but boosts generalization.
LogoDet-3K 3	160k images , 3,000 logo classes, wide variety of logos in natural scenes.	Provides scale and diversity. Risk: mismatch with our 86 F1 brands, but can be used for pretraining and hard negatives.
FlickrLogos-32 / Logos-32plus 4	∼8k images, 32 brands.	Adds clean annotations and background variety; manageable size makes it easy to integrate.
Synthetic Logo Compositions	Generated by pasting sponsor logos onto unlabeled F1 frames.	Balances class frequencies for underrepresented brands, though risks unrealistic artifacts if blending is poor.
Logo augmentation	Generated by using the sponsor logo dataset and modify the images through a variety of transformations, adjustments, or other techniques.	As F1 broadcasts capture sponsor logos from varied angles and with significant geometric distortions, robust logo recognition requires training data that reflects this real-world viewing experience. Humans will still be able to identify a logo if it is rotated, curved, or resized, but computer will have difficulty with this, thus we need to improve their capability to do so and maintain high accuracy when identifying a specific logo during race coverage.

Reasoning and trade-offs: The Roboflow dataset is directly relevant and clean, but modest in size. OpenLogo and LogoDet-3K offer scale and generalization at the cost of added complexity. Synthetic augmentations help address class imbalance but may produce unrealistic samples. Our strategy is to use Roboflow as the supervised core, with external datasets for pretraining and augmentation to mitigate its limitations.

3 Data-Processing Pipelines

We now describe both the training and inference pipelines, including ingestion, cleaning, transformations, and infrastructure considerations.

3.1 Data Schemas

Our train method centers on a primary dataset, Roboflow F1 Logo Dataset, from where we perform data augmentations and stratefied splitting, and potentially merge with other datasets. We designed a dataset schema that minimizes duplicated data while keeping dependencies and references clear.

Table 2: *Data schemas used in the training pipeline*

Entity	Schema / Description
Image file	JPEG/PNG under `s3://visight-data-yusufmoola/raw/roboflow/v8/{split}/images/` or `s3://visight-data-yusufmoola/processed/{version}/{split}/images/`.
YOLO label file	TXT per image under `.../labels/`, one line per bounding box: `class_id cx cy w h` (normalized).
Catalog (CSV/JSON)	Columns: `id` (hash of stem), `s3_img_path`, `s3_label_path`, `class_ids[]` (multi-hot), `split`. One per dataset version, acts as the index for training jobs.
Class map (YAML)	Canonical 86 F1 classes with IDs; synonym map maintained for external datasets (e.g., "Oracle Red Bull Racing" → "Oracle").
Training config (not yet implemented) (YAML)	Hyperparameters and augmentation config: `img_size`, `epochs`, `optimizer`, `lr`, `augment_probs`.
Run metadata (not yet implemented) (YAML)	`run.yaml`: code commit, random seeds, dataset hash, class map checksum, split ratios.
Validation report (not yet implemented)	Per-class counts, PR curves, and mAP@.5:.95 under `s3://visight-data-yusufmoola/stats/{run}/`.
Model artifacts (not yet implemented)	Model artifacts required for model portability and inference.

Catalog
Notice that the storage of the raw data itself (Image files, YOLO label files) and the metadata linking the data into datasets (Catalog, Class map) are **separate entities**. The catalog (CSV/JSON) acts as the index for training jobs, allowing Modal containers to reproducibly load images and labels from S3, perform stratified splitting, and log results. Each dataset version will have its own Catalog, which allows images to be reused in multiple datasets without added storage overhead. By separating raw assets, configuration, and run metadata, this schema ensures consistency, reproducibility, and ease of scaling across both training and inference pipelines.

YOLO structured image and label data
Each image is paired with a label file containing one line per bounding box in normalized YOLO format (`class_id cx cy w h`).

Run artifacts (not yet implemented)

- Run metrics: Our primary training evaluation metrics will be mAP@.5:.95 and per-class recall. We hope to also include system metrics like latency on GPUs and memory footprint and this may help guide our choice of inference runtime engine. Business proxies include error in share-of-voice compared to manual labels on broadcast clips. If Phase A suffices, we may stop there; otherwise we advance iteratively.
- Reproducibility and versioning artifacts: Each run saves run.yaml with seeds, hyperparameters, and class map checksums. Splits are deterministic and stored in the catalog. Future extensions may use DVC for reproducible snapshots.

Model artifacts for deployment (not yet implemented)
Artifacts and model cards (sources, hyperparameters, metrics, known failure cases) are published to S3 for AWS inference consumption.

- PyTorch weights: Retained to allow future fine-tuning or re-training. These are versioned and stored in S3 with associated metadata (dataset hash, split seed, hyperparameters).
- ONNX model (dynamic shapes): Provides an open, framework-agnostic representation that can run on CPUs and GPUs using ONNX Runtime. Dynamic shape support allows inference on different resolutions without retraining.
- TensorRT FP16 engine: Optimized for NVIDIA GPUs to achieve low-latency inference. FP16 quantization reduces memory and compute requirements while maintaining accuracy, enabling near real-time throughput on modest GPUs.

Label normalization and class map.
We define a fixed 86-class map of F1 sponsors as our source of truth. When incorporating additional datasets, class names

are normalized using a synonym map (e.g., "Oracle Red Bull Racing" → "Oracle"). This process prevents accidental class drift, keeps the detection head size stable, and ensures reproducibility across training runs. Currently, since we only have one brand dataset we are using, more complex fuzzy string matching logic are not yet implemented.

3.2 Infrastructure Considerations

Docker for consistent environment definition The inference pipeline runs in a containerized Docker environment for reproducibility across local development and AWS deployment. Currently, our Dockerfile installs system dependencies required for OpenCV, then copies the Python requirements and application code; after we have trained our model, it will also install the system dependencies needed to run the model. This guarantees dependency consistency across all environments and portability between local testing and cloud deployment. By containerizing early, we reduce potential deployment friction and ensure that the inference pipeline behaves identically regardless of where it is run.

Why S3 as a training data lake. While the final user workflow only involves uploading short broadcast clips for analysis, the training workflow is far heavier: we manage thousands of labeled images, augmented variants, synthetic samples, and multiple model checkpoints. Storing these directly in Modal would be brittle, so instead we use S3 as our data lake for a durable source of truth. S3 holds raw exports, processed splits, validation reports, and model artifacts in a versioned layout. This ensures reproducibility across experiments and team members. It also allows downstream inference services on AWS to directly consume the exported models and configuration files from the same repository, avoiding manual file transfers between training and serving environments.

(future) Modal integration. ETL and training jobs are executed on Modal in containerized environments preloaded with YOLO, augmentation libraries, and preprocessing scripts. Modal provides flexible GPU access for fine-tuning, synthetic generation, or pretraining sweeps, but its storage is ephemeral. To maintain persistence, all outputs—catalogs, stratified splits, augmented images, checkpoints, and logs—are automatically pushed to S3 at the end of each run. This division of labor keeps training scalable and low-cost (Modal) while ensuring serving is stable and always aligned with the latest exported artifacts (AWS). S3 acts as the bridge between these environments.

3.3 Pipeline 1: Data ingestion and catalog creation

Takes locally downloaded raw dataset and uploads it into s3, as well as produces a matching "raw" dataset catalog. The catalog generation ETL can also be reused for downstream processed datasets. This pipeline assumes that datasets are able to be downloaded locally, and is run on one's local machine as well.

1. Simple ingestion ETL (takes locally downloaded dataset from Roboflow and batch saves it into s3). It copies the train, test, and validation directories which each contain image and label directories, and a data.yaml file with contains the relative paths to each directory and a brand list. (pipelines/training_ingestion/upload_dataset.shin the repository.)
2. Catalog generation ETL: batch reads a dataset directory in s3 and summarizes the images, their splits, matching label file location. The catalog is as described in section 3.1. (pipelines/training_ingestion/image_brand_catalog.py)

Currently, we've run this pipeline to ingest the Roboflow F1 (primary) dataset, stratified split catalog, and data augmented catalog. All labels are normalized to YOLO format (class_id, cx, cy, w, h). Jobs were run on local systems and pushed outputs to S3 for consistent downstream use.

Requirements for correct behavior

- Raw datasets to ingest must have images, labels for each image, classes for each label, and a data.yaml file that summarizes the structure of the dataset.
- Dataset should already be grouped into at least one training split. Note that we makes another catalog if resplitting the same dataset (see section on stratefied splitting for an example).
- data.yaml should specify the location of each data split relative to the dataset directory. Each data split should have images and labels directories that store all the images and labels.
- Image path stems should be uniquely matching to their label.

When to run

- When ingesting a new dataset, use the entire pipeline to load it to the correct location in s3 and produce the matching raw data catalog.
- When generated a processed version of a raw or existing dataset, run the catalog generation ETL to create a catalog that can allow this dataset to be seemlessly used for training.

3.4 Pipeline 2: Raw dataset processing (cleaning, resampling, augmenting)

Stratified sampling and split mechanics.
Because images contain multiple brands, naive random splitting can under-represent rare classes. We use iterative multi-label stratification:

1. Construct a multi-hot target matrix $Y \in \{0,1\}^{N \times C}$ where $Y[i,c] = 1$ if image i contains class c.
2. Compute desired per-class counts for train/val/test (75/15/10).
3. Sort classes by rarity and greedily assign images to minimize distribution error while maintaining per-class balance.

We validate splits by checking per-class counts, ensuring no duplicate image hashes. This stratified split pipeline functions as a reusable ETL stage: it extracts image–label pairs from the raw Roboflow export, transforms them into balanced train/val/test partitions using multi-label stratification, and loads both the manifest and class count statistics into S3. Because the process is parameterized (ratios, seeds, output paths), it can be rerun consistently across new dataset versions, ensuring reproducible preprocessing and a standardized input format for downstream training.

Data augmentation
To address the long-tail distribution of our dataset and improve the robustness of our detector, we performed offline pre-training augmentations specifically on under-represented classes. The augmentation pipeline included photometric adjustments (brightness/contrast, hue, saturation), realistic broadcast degradations (blur, compression artifacts, downscaling, noise), mild geometric transformations (affine shifts, small rotations, perspective warps, random scaling), and occasional occlusions (coarse dropout). These operations were chosen to reflect the conditions logos are likely to appear in during real F1 footage, such as motion blur, low resolution, or partial occlusion, while maintaining label consistency. By augmenting only minority classes, we increased class balance and improved model exposure to harder cases. Importantly, this offline step complements the online augmentations that YOLO applies during training (e.g., mosaic, flips, color jitter), ensuring the model benefits from balanced exposure and diverse real-time changes.

Pre-processing summary
The raw dataset had imbalances with classes missing entirely in validation and test, and KL divergence indicating distribution drift across splits. Stratified sampling greatly improved coverage (reducing zero-class splits to nearly zero) and aligned each split's class distribution with the global dataset. Augmentation further increased representation of long-tail training classes, although validation/test distributions remain slightly divergent since augmentation was not applied there. Together, stratification ensures fair evaluation, while augmentation enhances robustness during training. In addition, the total image counts chart shows that while stratification redistributed samples more evenly across splits, augmentation primarily increased the size of the training set by oversampling minority classes, leaving validation and test set sizes unchanged. This reinforces that augmentation was used only to balance and enrich the training distribution, while evaluation splits remain untouched to provide a fair benchmark of model generalization.

As shown in Appendix A Figure 4, stratification eliminates most zero-class splits. The improved distribution alignment is evident in Figure 5, and the effect on split sizes is summarized in Figure 3.

3.5 Pipeline 3: Inference (video ingestion and labeling)

The inference pipeline process broadcast and highlight video clips to detect and quantify brand logo appearances. It accepts a video file, extracts frames at configurable FPS, runs model inference to identify logos with bounding boxes and confidence scores, and generates structured results. The inference pipeline is to be run for post-training validation on held-out clips to benchmark performance and for production inference when uploading race footage.

3.6 Evolving Data Needs

We adopt a staged training approach so we can (a) get a working baseline quickly, (b) diagnose failure modes with data, and (c) add complexity only where it moves the needle. Each phase has different dataset needs.

- **Phase A — Baseline (Roboflow-only):** Clean, validate, stratify, split, fine-tune YOLO on the Roboflow F1 dataset. This establishes reference accuracy, recall and per-class performance.
- **Phase B — Generic logo pretraining (OpenLogo, LogoDet-3K):** Pretrain the detector backbone to learn "logo-ness" on large, heterogeneous corpora; then replace the head and fine-tune on F1. This targets generalization and recall, especially for small and blurred logos.
- **Phase C — Hard negatives and synthetic upsampling:** Reduce broadcast false positives using non-F1 logos and background frames; increase tail-class coverage via copy-paste synthesis with official logo assets.

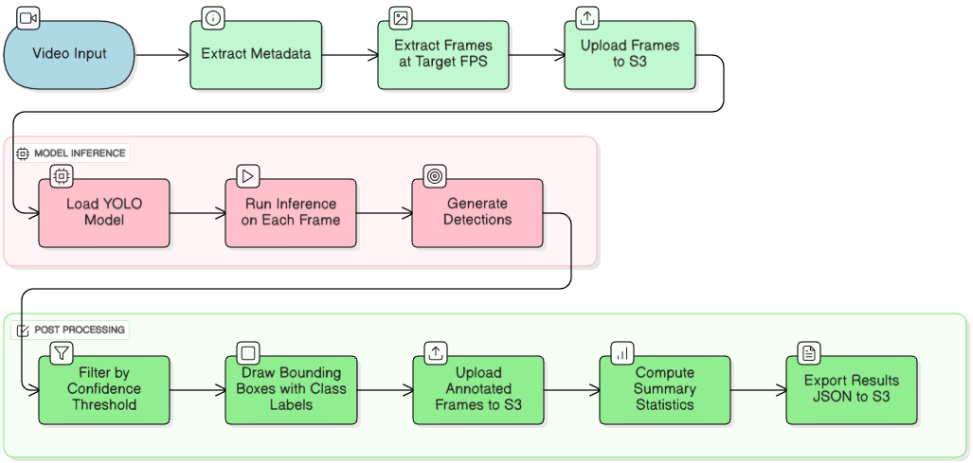

Figure 2: Inference Pipeline. The input video is processed frame-by-frame. Each frame is passed through the model, which outputs coordinates of bounding boxes for logos on the frame. We keep the bounding boxes above a specific confidence threshold, draw them on the frame, and upload back to S3.

- **Phase D — Final fine-tune and export:** Performed after whichever prior phase we stop at, this consolidates the model with tuned augmentation probabilities and produces deployable artifacts (PyTorch, ONNX, TensorRT). Each phase is gated by metrics (mAP@.5:.95, per-class recall, false-positive rate on broadcast clips). We advance only if thresholds are not met, but Phase D always occurs at the end of the chosen training path.

References

1. Roboflow. (2025). *F1 Logos Dataset*. Available at: https://universe.roboflow.com/akip07/f1-logos-k0vsh

2. Su, Z., Zhu, H., Zheng, Q., et al. (2019). *Open Logo Detection Challenge*. Queen Mary University of London. Dataset available at: https://qmul-openlogo.github.io/

3. Wang, J., Song, J., Yang, X., et al. (2020). *LogoDet-3K: Large-Scale Logo Detection Dataset*. Dataset available at: https://github.com/Wangjing1551/LogoDet-3K-Dataset

4. Kalantidis, Y., Pueyo, L., Avrithis, Y., & Saavedra, J. (2011). *FlickrLogos-32: A New Dataset for Scalable Logo Recognition*. Available at: https://www.uni-augsburg.de/en/fakultaet/fai/informatik/prof/mmc/research/datensatze/flickrlogos/

A Additional Data Diagnostics

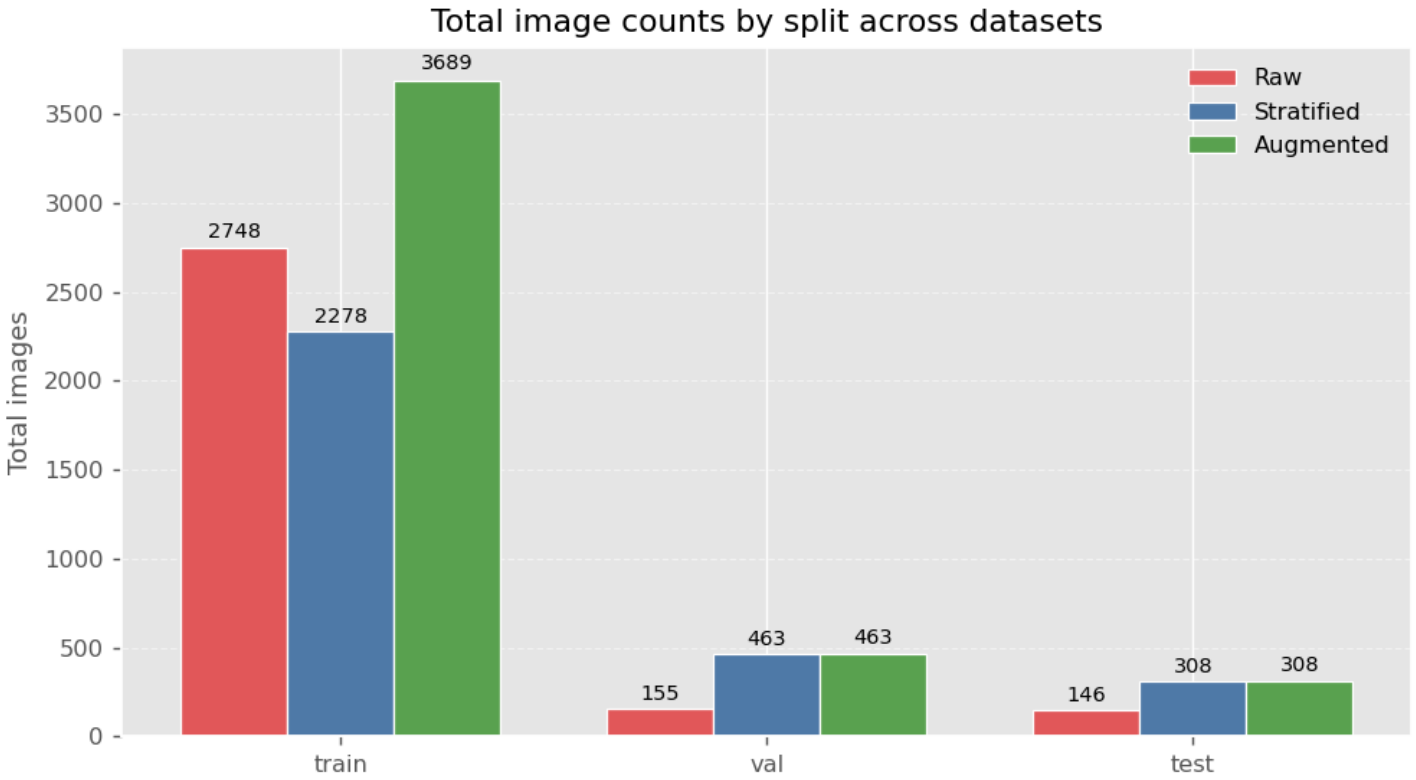

Figure 3: Total image counts by split across datasets. Augmentation increases training set size while validation/test remain unchanged.

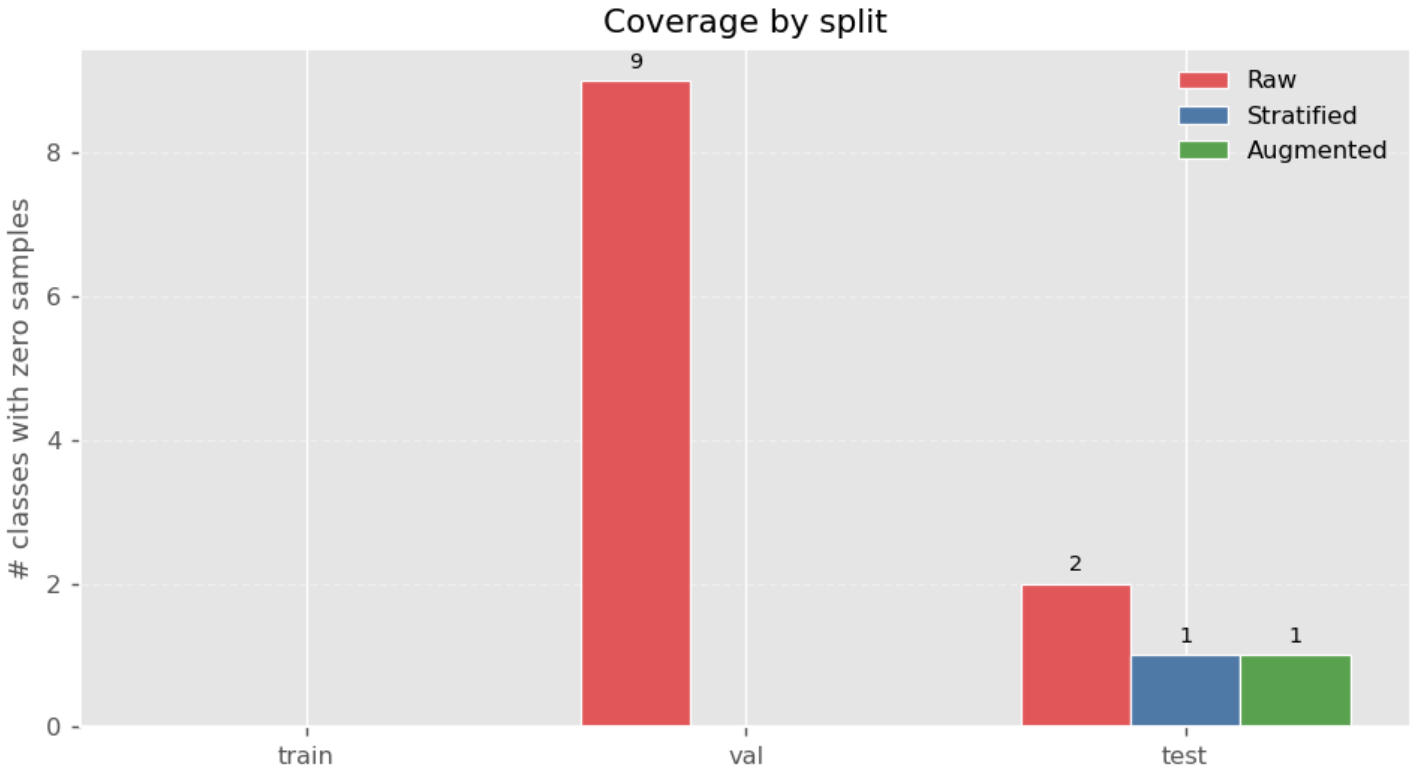

Figure 4: Coverage by split: number of classes with zero samples in train/val/test for Raw, Stratified, and Augmented datasets (lower is better).

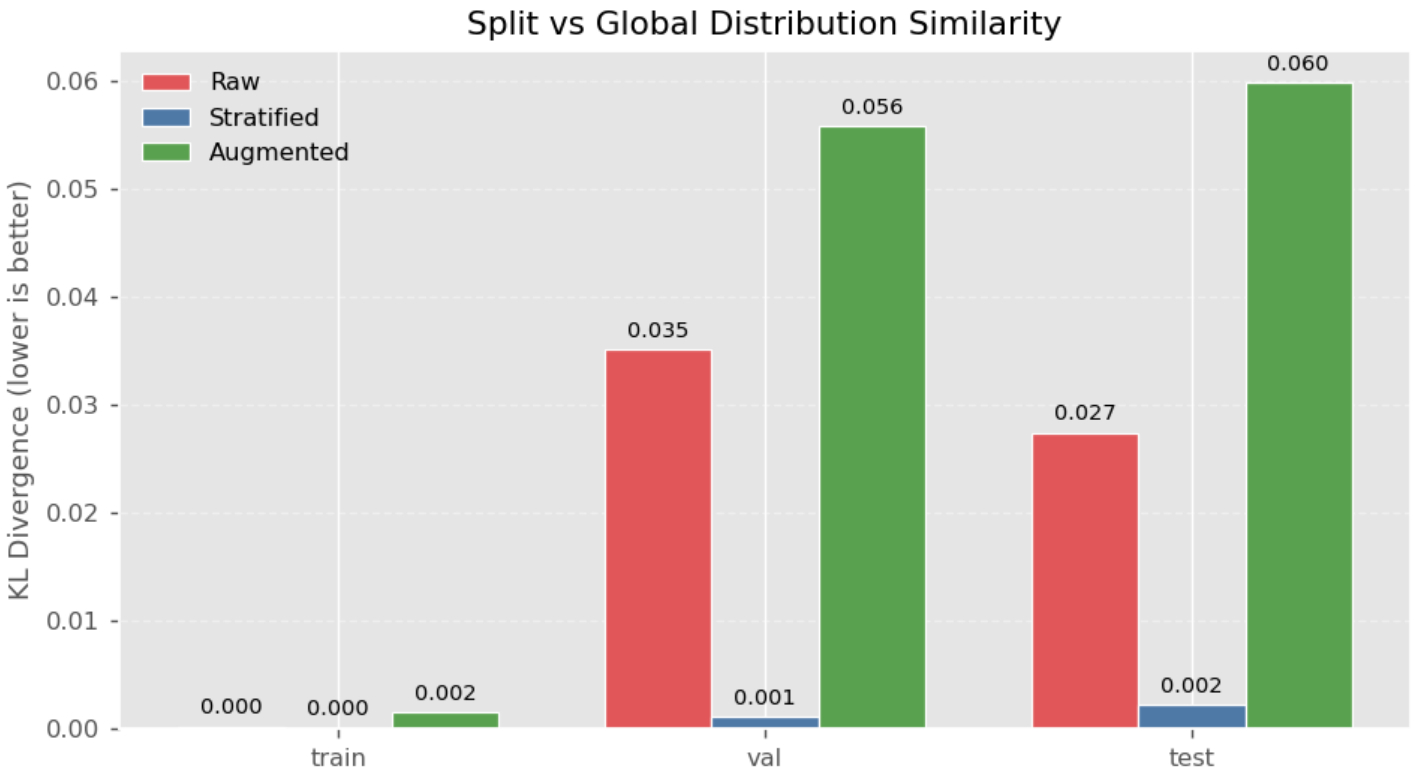

Figure 5: Split vs. global distribution similarity (KL divergence). Lower values indicate splits that better match the overall class distribution.